ay PostgreSQL Course

Your First SELECT

eek 1: SQL Foundations

Duration	1 Hour
Level	Complete Beginner
Goal	Write SELECT queries and alias columns
Week	Week 1 — SQL Foundations
Database	PostgreSQL 16 on Neon
Notebook	day_02_first_select.ipynb

✦ What You Will Learn Today

• Write a basic SELECT statement with the correct syntax • Use SELECT * to inspect a table (and when NOT to use it in pipelines) • Select specific columns by name • Rename columns using the AS keyword (column aliasing) • Create computed (derived) columns directly in your SELECT • Understand how SELECT fits into a real data engineering pipeline

Session Agenda

Time	Segment	Activity
00:00–05:00	Welcome & Setup	Connect Jupyter to Neon, verify table exists
05:00–15:00	The SELECT Statement	Syntax, semicolons, SELECT *, specific columns
15:00–30:00	Column Aliasing with AS	Renaming columns, quoting, computed columns
30:00–45:00	Live Coding — Pipeline Scenarios	Build a realistic pipeline SELECT step by step
45:00–55:00	Hands-On Exercise	Students write their own query independently
55:00–60:00	Recap & Day 3 Preview	Summary + intro to WHERE clause filtering

00:00 – 05:00

WELCOME & SETUP

Segment 1 — Welcome & Setup

Welcome back! Yesterday you installed PostgreSQL and understood what a relational database is. Today we write actual SQL for the first time. By the end of this hour you will have written five different SELECT queries against a real table on Neon.

Camera Script:
"Open your Jupyter notebook day_02_first_select.ipynb. In Cell 2, paste your Neon connection string and run it. You should see: Connected: postgresql://... — if you see that, you're good to go."

```
# Cell 2 — paste and run this:

%load_ext sql

%sql postgresql+psycopg2://USER:PASS@ep-xxxx.neon.tech/neondb?sslmode=require

# Then run Cell 3 to create the orders table and insert 10 rows.
```

05:00 – 15:00

THE SELECT STATEMENT

Segment 2 — The SELECT Statement

A SELECT statement is how you ask PostgreSQL for data. Think of it as a question: 'PostgreSQL, give me these columns from this table.' The database finds the matching data and returns it as a result set — a grid of rows and columns.

The anatomy of a SELECT statement:

```
SELECT

FROM ;

-- The semicolon ; marks the END of the statement.

-- Without it, PostgreSQL waits for more input.
```

Key syntax rules to cover on camera:

Rule	Detail
Semicolon ;	Every statement ends with ; — PostgreSQL won't execute without it
UPPERCASE keywords	SQL keywords (SELECT, FROM) are case-insensitive, but uppercase is convention
Column names	Lowercase table/column names are standard in PostgreSQL
Whitespace	Extra spaces and newlines are ignored — format for readability

SELECT * — All Columns

```
%%sql

SELECT *

FROM orders;

-- Returns ALL 6 columns: order_id, customer, product,

-- quantity, unit_price, order_date
```

■ Pipeline Tip

In production pipelines, NEVER use SELECT * in your final query. If someone adds a new column to the source table, SELECT * silently pulls it in and can break downstream systems that expect a fixed schema. Always name your columns explicitly.

SELECT Specific Columns

```
%%sql

SELECT order_id, customer, product, unit_price

FROM orders;

-- Only 4 columns returned, in the order we specified.

-- Column order in output matches SELECT list, not the table.
```

15:00 – 30:00

COLUMN ALIASING WITH AS

Segment 3 — Column Aliasing with AS

The AS keyword lets you rename a column in your query output. The original table is not changed — only the label in the result set. This is one of the most-used features in pipeline SQL because source column names rarely match what the destination system expects.

Syntax:

```
SELECT column_name AS alias_name

FROM table_name;

-- Aliases with spaces need double quotes:

SELECT column_name AS "Alias With Spaces"

FROM table_name;
```

Pipeline example — renaming for the destination schema:

```
%%sql
SELECT
order_id AS id,
customer AS customer_name,
product AS product_name,
unit_price AS price_inr
FROM orders;
```

Computed (Derived) Columns:

You can compute a new value directly in SELECT and give it a name. The column does not exist in the table — it is derived on the fly.

```
%%sql
SELECT
order_id,
customer,
quantity,
unit_price,
quantity * unit_price AS total_amount
FROM orders;
```

■ Pipeline Tip

In data pipelines, computed columns like total_amount = quantity * unit_price are transformation logic. In dbt (a popular data transformation tool), this same SQL lives inside a model file — everything you learn today transfers directly.

30:00 – 45:00

LIVE CODING — PIPELINE SCENARIOS

Segment 4 — Live Coding: Pipeline Scenarios

Walk through building a realistic pipeline SELECT step by step on camera. Narrate your thinking — explain WHY you are choosing each column and alias.

Scenario A — Staging table inspection

```
%%sql
SELECT *
```

```
FROM orders;

-- "Let me see what landed in this staging table"
```

Scenario B — Pick columns for a downstream report

```
%%sql

SELECT

order_id AS id,

customer AS customer_name,

product,

order_date AS placed_on

FROM orders;
```

Scenario C — Full transformation — rename + compute

```
%%sql

SELECT

order_id AS id,

customer AS customer_name,

product,

quantity,

unit_price AS price_inr,

quantity * unit_price AS total_value,

order_date AS placed_on

FROM orders;
```

45:00 – 55:00

HANDS-ON EXERCISE

Segment 5 — Hands-On Exercise

✏■ Exercise — Build a Pipeline SELECT Query

Using the orders table, write a single SELECT query that does ALL of the following: 1. Retrieves: order_id, customer, product, quantity, unit_price, order_date 2. Aliases order_id → id 3. Aliases unit_price → price_inr 4. Adds a computed column: total_value = quantity * unit_price 5. Aliases order_date → placed_on Expected output columns: id, customer, product, quantity, price_inr, placed_on, total_value

Expected Output (10 rows, 7 columns):

id	customer	product	quantity	price_inr	placed_on	total_value
1001	Alice Sharma	Laptop	1	75000.00	2024-01-15	75000.00
1002	Bob Mehta	Mouse	3	499.00	2024-01-17	1497.00
...	...	...	...	...	...	...

Common Errors & Fixes:

Error	Cause	Fix
column "order_id" does not exist	Typo in column name	Check spelling with SELECT * first
syntax error at or near AS	Missing comma before AS	Add comma: unit_price, order_date AS placed_on
ERROR: syntax error at end of input	Missing semicolon ;	End statement with ;
column "total_value" does not exist	Using alias in WHERE (Day 3+)	Aliases can't be used in WHERE — use the expression

Solution:

```
%%sql
SELECT
order_id AS id,
customer,
product,
quantity,
unit_price AS price_inr,
order_date AS placed_on,
quantity * unit_price AS total_value
FROM orders;
```

55:00 – 60:00

RECAP & DAY 3 PREVIEW

Segment 6 — Recap & Day 3 Preview

■	`SELECT * FROM table`	Fetch all columns — inspection only, never in production pipelines
■	`SELECT col1, col2 FROM table`	Fetch specific columns — always name them explicitly
■	`col AS alias`	Rename a column in the output — table is unchanged
■	`expr AS alias`	Compute a derived field and give it a name
■	`SELECT without FROM`	Test expressions without needing a table

Day 3 Preview — Camera Script:

> *"Right now our queries return every single row in the table. In a real pipeline with millions of rows, you never want to pull everything. Tomorrow we add the WHERE clause — the filter. You'll learn to say: give me only orders placed after January 1st, or only rows where the product is a Laptop, or only rows where the customer email is missing. That's Day 3 — see you then."*

Appendix — Day 2 Quick Reference Card

Pattern	Example	Notes
`SELECT *`	`SELECT * FROM orders;`	All columns — inspection only
`SELECT specific cols`	`SELECT order_id, customer FROM orders;`	Name what you need
`Column alias`	`SELECT unit_price AS price_inr ...`	Rename output column
`Alias with spaces`	`SELECT order_id AS "Order ID" ...`	Double quotes for spaces
`Computed column`	`SELECT qty * price AS total ...`	Math in SELECT
`SELECT without FROM`	`SELECT CURRENT_DATE AS today;`	Test expressions
`Multiple aliases`	`SELECT a AS x, b AS y, c AS z ...`	Alias as many as needed

The Orders Table Used in This Session:

Column	Type	Example Value
`order_id`	`INTEGER`	`1001`
`customer`	`VARCHAR(100)`	`'Alice Sharma'`
`product`	`VARCHAR(100)`	`'Laptop'`
`quantity`	`INTEGER`	`1`
`unit_price`	`NUMERIC(10,2)`	`75000.00`
`order_date`	`DATE`	`2024-01-15`